As a DevOps Architect, I have designed the following infrastructure for a **Banking Management System**. Given the sensitive nature of financial data, this architecture prioritizes **security, high availability, and auditability**.

---

1. Deployment Architecture

- **Pattern:** Microservices architecture deployed on **Kubernetes (EKS/GKE)**.
- **Network:** Multi-AZ (Availability Zone) deployment.

* **Public Subnet:** Load Balancer (WAF-protected).

* **Private Subnet:** Application pods (Backend), Redis cache, and Database.

- **Security:** Mutual TLS (mTLS) between services via **Istio Service Mesh**.

2. Docker

- **Base Image:** `distroless/java` or `alpine` (minimal footprint reduces attack surface).
- **Multi-stage builds:**

* Stage 1: Build (Maven/Gradle).

* Stage 2: Package (Copy artifacts to runtime image).

- **Security Scanning:** Images scanned by **Trivy** in the CI pipeline.

3. CI/CD (GitHub Actions or GitLab CI)

- **Pipeline Stages:**

1. **Code Quality:** SonarQube (Static Analysis).

2. **Security:** Snyk/OWASP Dependency Check.

3. **Build:** Push Docker image to Private Container Registry (ECR/GCR).

4. **CD: ArgoCD** (GitOps approach). The cluster pulls changes from the Git repo, ensuring the desired state always matches the actual state.

4. Hosting

- **Cloud Provider:** AWS (Recommended for strict compliance).

- **Orchestration:** Managed Kubernetes (EKS).
- **Edge/WAF:** AWS WAF + CloudFront for DDoS protection and geographic filtering.

5. Database Hosting

- **Engine:** PostgreSQL (RDS Multi-AZ).
- **Compliance:** Encrypted at rest (AWS KMS) and in transit (SSL/TLS).
- **Read Replicas:** Used for reporting and heavy query offloading.
- **Security:** Database is never exposed to the public internet; accessible only via private VPC endpoints.

6. Environment Variables

- **Secret Management: HashiCorp Vault** or **AWS Secrets Manager**.
- *Do not store secrets in K8s ConfigMaps.* Use **External Secrets Operator** to inject secrets into pods as environment variables or memory-mounted files at runtime.

7. Monitoring

- **Stack: Prometheus + Grafana**.
- **Metrics:**

* Transaction success rates (Golden Signals).

* Database connection pool saturation.

* Latency of inter-service calls.

- **Alerting:** AlertManager connected to PagerDuty or Slack.

8. Logging

- **Strategy:** ELK Stack (Elasticsearch, Logstash, Kibana) or **Loki/Grafana**.
- **Compliance Requirement:** Logs must be immutable. Use **S3 Object Lock** for long-term audit logs (PCI-DSS requirement).
- **Distributed Tracing: Jaeger** (essential for debugging transaction failures across microservices).

9. Scaling

- **Horizontal Pod Autoscaler (HPA):** Scales pods based on CPU/Memory usage.
- **Cluster Autoscaler:** Provisions new nodes when pods cannot be scheduled.
- **Database:** Proxy layer (e.g., **PgBouncer**) to manage connection pooling during traffic spikes.

10. Backup Strategy

- **Database:**

* Automated daily snapshots with 30-day retention.

* Point-in-time recovery (PITR) enabled for granular recovery in case of accidental data corruption.

- **Disaster Recovery (DR):**

* Cross-region replication of DB snapshots.

* Infrastructure as Code (Terraform) scripts ready to deploy the entire environment in a secondary region.

- **Testing:** Quarterly "Restore Drills" to verify data integrity.

Critical Considerations for Banking:

1. **PCI-DSS:** Ensure the network architecture isolates the Payment Application from the rest of the environment.
2. **Audit Trail:** Every state-changing request must be logged with a correlation ID, user context, and timestamp.
3. **Rate Limiting:** Implement at the API Gateway level to prevent brute-force attacks on financial endpoints.